

Evaluation Strategy for Text-to-SQL Agent System

1. Introduction

A Text-to-SQL system converts natural language questions into executable SQL queries over a relational database. In this project, the system is extended into an agentic pipeline that not only generates SQL but also executes it and returns final answers.

Evaluating such a system is critical because correctness cannot be measured only by syntactic validity. The system must also produce **semantically correct results** aligned with user intent.

The goal of this evaluation framework is to define structured metrics and procedures to measure the performance, reliability, and robustness of a Text-to-SQL agent.

2. Evaluation Objectives

The system will be evaluated based on the following objectives:

- Correct interpretation of natural language queries
- Accurate SQL generation aligned with schema
- Correct execution on PostgreSQL database
- Robust handling of errors and retries
- Consistency of final output results
- Performance efficiency (latency and execution time)

3. Evaluation Metrics

3.1 SQL Correctness

Measures whether the generated SQL is logically and syntactically correct.

- Valid table names
- Valid column names

- Correct JOIN conditions
- Proper aggregation logic

3.2 Execution Success Rate

Percentage of queries that execute successfully without runtime errors.

Formula:

$$\text{Execution Success Rate} = (\text{Successful Executions} / \text{Total Queries}) \times 100$$

3.3 Result Accuracy

Compares output results of generated SQL against ground truth SQL results.

- Exact match preferred
- Partial match allowed for unordered outputs
- Aggregated values must match exactly

3.4 Schema Alignment Accuracy

Evaluates whether the correct schema elements are used:

- Correct tables selected
- Correct columns selected
- Proper foreign key usage

3.5 Join Accuracy

Checks whether relationships between tables are correctly identified.

- Correct ON conditions
- No missing or incorrect joins
- No unnecessary joins

3.6 Aggregation Accuracy

Evaluates correctness of:

- COUNT
- SUM
- AVG
- MIN / MAX
- GROUP BY logic

3.7 Retry and Self-Correction Performance

Measures system robustness when errors occur.

- Number of failed queries
- Number of successful retries
- Error recovery rate

Formula:

Retry Success Rate = (Fixed Queries After Retry / Failed Queries) × 100

3.8 Query Execution Latency

Measures time taken to:

- Generate SQL
- Execute query
- Return result

Important for real-world scalability.

3.9 Natural Language Response Quality

Evaluates how well the system explains results.

Criteria:

- Clarity
- Correctness
- Human readability

4. Benchmark Dataset

A benchmark dataset of 60(50 given, 10 self created) structured SQL questions is used for evaluation.

Each entry contains:

- Natural language question
- Ground truth SQL
- Intent
- Tables involved
- Columns involved
- Joins and filters

This dataset is used as the gold standard for comparison.

5. Evaluation Pipeline

The system evaluation follows this pipeline:

1. Input natural language question
2. Generate SQL query
3. Validate SQL syntax and schema
4. Execute SQL on PostgreSQL
5. Compare results with expected output
6. Log success or failure
7. Apply retry mechanism if needed
8. Record final evaluation score

6. Error Handling Evaluation

The system must be tested against common failure cases:

- Missing column errors
- Invalid table names
- Incorrect join conditions

- Syntax errors
- Empty result sets

The agent is expected to:

- Detect errors from PostgreSQL response
- Correct SQL automatically
- Retry execution (maximum 1–3 attempts depending on system design)

7. Overall System Score

Final system performance can be summarized using weighted scoring:

- SQL correctness: 25%
- Execution success: 20%
- Result accuracy: 25%
- Join and schema correctness: 15%
- Retry performance: 10%
- Response quality: 5%

8. Conclusion

This evaluation framework ensures that the Text-to-SQL system is not only syntactically correct but also semantically reliable and production-ready.

It provides a structured way to measure performance across multiple dimensions, making it suitable for benchmarking and iterative improvement of agentic SQL systems.